

Received 31 January 2026, accepted 13 February 2026, date of publication 18 February 2026, date of current version 25 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3665980

RESEARCH ARTICLE

A Secure and Efficient LLM-Based System for Natural Language Query-to-SQL Translation in IoT Data Management

MARIO SAN EMETERIO DE LA PARTE^{ID}, YITING WANG^{ID}, JOSÉ-FERNÁN MARTÍNEZ-ORTEGA^{ID},
AND NÉSTOR LUCAS MARTÍNEZ^{ID}

Group of Next Generation Networks and Services (GRyS), Departamento de Ingeniería Telemática y Electrónica (DTE), Escuela Técnica Superior de Ingeniería y Sistemas de Telecomunicación, Universidad Politécnica de Madrid (UPM), 28031 Madrid, Spain

Corresponding author: Mario San Emeterio de la Parte (mario.sanemeterio@upm.es)

This work was supported by the Horizon 2020 DEMETER Project, from INDUSTRIAL LEADERSHIP—Leadership in Enabling and Industrial Technologies—Information and Communication Technologies (ICT), funded by European Union's Horizon 2020 Research and Innovation Program, and Italy, Luxembourg, Spain, Germany, Greece, United Kingdom, Norway, Czechia, Montenegro, Belgium, Romania, Ireland, Finland, Serbia, Portugal, Poland, Georgia, and Slovenia, under Grant 857202.

ABSTRACT Structured data embodies vast reservoirs of actionable knowledge, yet non-technical users often lack the tools to effectively access or leverage these resources. This paper presents a novel middleware system architecture that harnesses Large Language Models (LLMs) to translate natural language queries (NLQs) into executable SQL statements, enabling seamless interaction with relational databases without requiring technical expertise. To bridge a critical gap in existing research, the proposed system integrates a robust security framework featuring multi-factor authentication and fine-grained, role-based access control. A comprehensive evaluation was conducted using over 813,000 real spatio-temporal IoT samples from the H2020-AFarCloud international project, comparing both general-purpose LLMs and models fine-tuned for NLQ-to-SQL translation. Two complementary evaluation metrics—Execution Accuracy (EX) and Valid Efficiency Score (VES)—were employed to assess syntactic correctness, semantic fidelity, and runtime efficiency. Experimental results show that GPT-o3 mini-high achieves the highest overall performance in terms of average VES, while DeepSeek R1 exhibits the most consistent behavior across heterogeneous query scenarios, as reflected by lower performance variability. These findings underscore the interplay between model capability and computational efficiency, demonstrating the potential of LLM-driven middleware to democratize structured data access and enhance data-driven decision-making.

INDEX TERMS Database management systems (DBMS), artificial intelligence (AI), data engineering, data science, human-computer interaction (HCI), human machine interface (HMI), Internet of Things (IoT), large language models (LLMs), natural language processing (NLP).

I. INTRODUCTION

The increasing availability of data generated by interconnected devices has intensified the need for accessible and efficient mechanisms to exploit relational databases, particularly in Internet of Things (IoT) environments. In such contexts, stakeholders such as farmers, industrial operators, and domain analysts often rely on timely and accurate information for decision-making but lack the technical

expertise required to formulate complex SQL queries. Natural Language Interfaces to Databases (NLIDBs) aim to bridge this gap by enabling users to interact with data repositories using natural language queries (NLQs).

In response to this challenge, this paper proposes a Large Language Model (LLM)-based system architecture designed to translate natural language queries into SQL queries (Text-to-SQL), enabling seamless database access for non-technical users. Acting as an intermediary between the user and the database, the proposed system interprets the intent behind natural-language inputs and converts it into syntactically and

The associate editor coordinating the review of this manuscript and approving it for publication was Xianzhi Wang^{ID}.

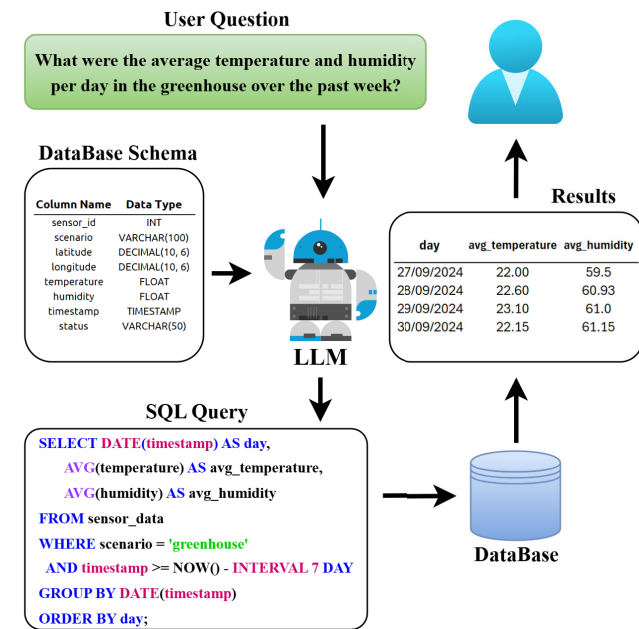


FIGURE 1. Proposed NLQ-to-SQL through (LLM)-based middleware.

semantically valid SQL queries capable of retrieving the requested data (see Figure 1). By abstracting away the complexities of query formulation, the system significantly lowers the barrier to data access and enables a broader range of users to benefit from data-driven decision-making processes.

Recent advances in LLMs have substantially improved the performance of NLDBs, especially for Text-to-SQL translation tasks. Modern LLMs demonstrate strong capabilities in mapping NLQs to syntactically valid SQL queries and adapting to diverse database schemas. As a result, LLM-based Text-to-SQL systems have attracted considerable attention and have been successfully applied across multiple domains [1]. Early deep learning approaches, such as Seq2SQL [2], pioneered neural Text-to-SQL translation, while more recent LLM-based methods have further improved generalization and adaptability. However, most existing approaches primarily focus on translation accuracy and overlook critical system-level requirements that arise in real-world deployments.

One such requirement is access control and security enforcement. In practical IoT scenarios, databases typically store sensitive, multi-tenant data subject to strict authorization policies. Users are often restricted to specific subsets of the data based on their roles, identities, or operational contexts. This aspect is particularly critical in IoT environments, where the number of connected devices is expected to grow from approximately 40 billion in 2023 to 49 billion by 2026, with a compound annual growth rate of 7%, as reported by the International Data Corporation [3]. Despite this, the majority of LLM-based Text-to-SQL systems implicitly assume unrestricted database access or rely on external mechanisms to filter results after query execution. This assumption poses significant risks, as generated SQL queries

may inadvertently expose unauthorized data or violate access control constraints.

This work addresses this gap by investigating the integration of security-aware access control mechanisms directly into the NLQ-to-SQL translation pipeline. Rather than treating security as an external post-processing step, the proposed approach incorporates role-based access control (RBAC) constraints into the query generation workflow. The high-level translation process is explicitly decomposed into two stages: (i) the generation of a semantically correct Core SQL query from the NLQ, and (ii) a policy-aware SQL rewriting stage that enforces access control rules based on the current user context without altering the core query logic. This separation enables security constraints to be applied systematically while preserving the expressiveness and accuracy of the generated SQL.

In addition to security considerations, the evaluation of Text-to-SQL systems in real-world settings requires metrics that capture not only execution correctness but also query efficiency. Existing evaluation approaches often rely solely on exact-match or execution accuracy metrics, which fail to account for the computational cost of generated queries. To address this limitation, this work adopts and extends the Valid Efficiency Score (VES), a metric that jointly evaluates execution correctness and relative execution efficiency with respect to expert-defined reference queries. This allows a more comprehensive assessment of model behavior under realistic operational constraints.

The proposed framework is evaluated using a diverse set of NLQs derived from a real IoT-based monitoring scenario. Multiple state-of-the-art LLMs are assessed under a controlled and reproducible experimental setup, considering both translation quality and consistency across heterogeneous queries. Furthermore, a preliminary security behavior validation is conducted to empirically demonstrate the effectiveness of the proposed policy-aware mechanism in preventing unauthorized data access and privilege escalation.

The main contributions of this work can be summarized as follows:

- The design of a security-integrated NLQ-to-SQL architecture that combines LLM-based translation with explicit role-based access control, tailored to IoT environments.
- The adoption and application of the Valid Efficiency Score (VES) as a unified metric to evaluate both correctness and efficiency of generated SQL queries.
- An extensive experimental evaluation across multiple LLMs, including an analysis of performance consistency and a preliminary empirical validation of security-related behaviors.

By jointly addressing translation accuracy, efficiency, and security within a unified framework, this work contributes toward the development of reliable, secure, and user-friendly NLQ-to-SQL systems suitable for deployment in real-world IoT data management scenarios.

The proposed solution aligns with several Sustainable Development Goals (SDG) of the United Nations [4], including SDG 9 on industry, innovation, and infrastructure, and SDG 2 on sustainable agriculture. By democratizing access to IoT-generated data and enabling non-technical users to interact with complex databases through natural language, the system contributes to more inclusive, efficient, and sustainable data-driven decision-making processes.

The remainder of the paper is organized as follows. Section II outlines the research objectives. Section III reviews related work on Text-to-SQL and LLM-based approaches. Section IV describes the proposed system architecture and its security mechanisms. Section V presents the experimental evaluation and analysis, while Section VI discusses future research directions. Finally, Section VII concludes the paper.

II. RESEARCH OBJECTIVES

The principal objective of this work is to democratize access to structured data for users without technical training, thereby eliminating the barrier imposed by traditional query languages and enabling more informed, data-driven decisions. To achieve this, the paper introduces an LLM-based system that automatically translates user-formulated natural language queries into syntactically and semantically valid SQL statements, which are then executed against a relational database. By abstracting away the need for SQL proficiency, the system empowers non-technical stakeholders to retrieve and analyze IoT-generated data with minimal friction.

To support these objectives, the article provides a comprehensive exposition of the proposed solution, detailing its design and architectural components, including an integrated security framework that provides robust authentication and fine-grained authorization mechanisms. This ensures secure role-based access control at both table and field levels, positioning the system as a comprehensive and innovative solution.

Furthermore, the article presents an empirical evaluation performed on a real-world IoT time-series dataset from the H2020-AFarCloud [5] precision agriculture project. The evaluation considers a diverse set of LLMs. These include both generic-context models and models developed specifically for natural language-to-SQL translation. Two key metrics are employed: Execution Accuracy (EX), to assess the syntactic and semantic correctness of the generated SQL queries; and Valid Efficiency Score (VES) [6], to measure execution efficiency in terms of query runtime. Together, these metrics enable the identification of LLMs capable of producing valid, semantically accurate queries while maintaining acceptable performance for integration into the proposed system.

III. RELATED WORK

The evolution of Text-to-SQL approaches has progressed through several key stages, from early rule-based systems to deep learning and now LLMs.

Initially, Text-to-SQL systems primarily relied on rule-based methods [7], [8], where predefined templates and rules were used to map NL questions into SQL statements. While effective for domain-specific tasks, rule-based approaches struggled with handling ambiguous language, complex query structures, and adaptation to new domains, requiring extensive manual effort to create and maintain the rules.

With the rise of deep learning, approaches based on Deep Neural Networks (DNNs) have emerged [9]. Sequence-to-Sequence (Seq2Seq) is a well-known DNN-based method that maps NL questions to SQL queries using an encoder-decoder framework [10], [11] [12]. The encoder generates vector representations of the input and table schema, while the decoder converts these representations into a corresponding SQL query. Graph Neural Networks (GNNs) extend this approach by representing the database schema as a graph, capturing complex relationships between tables and columns to enhance schema representation and improve SQL generation accuracy [13].

Pre-trained Language Model (PLM)-based approaches [14], [15] [16] leverage PLMs trained on large-scale text corpora (e.g., BERT [17], T5 [18]) and fine-tune them on smaller datasets for specific text-to-SQL tasks [9]. RESDSQL [16] extends Seq2Seq PLMs (e.g., T5) by introducing ranking-enhanced encoding, which filters and ranks relevant schema items before encoding, and skeleton-aware decoding, which first generates a SQL skeleton before filling in details, thereby improving schema understanding and SQL generation performance.

As pre-trained models and training data scale, PLMs have evolved into LLMs. The LLMs excel at understanding non-expert user questions and possess strong code generation capabilities [19], requiring minimal prompt design to achieve desired results [20]. They have become state-of-the-art tools in Text-to-SQL tasks. The rise of LLMs has led to two main approaches: prompt engineering and fine-tuning. The former leverages techniques such as Retrieval-Augmented Generation (RAG) for knowledge retrieval, few-shot and zero-shot learning for demonstrations, and Chain-of-Thought (CoT) for reasoning [21], guiding the LLM in generating SQL queries without the need for explicit retraining or fine-tuning. The latter adapts pre-trained LLMs to specific Text-to-SQL datasets. ChatGPT's zero-shot ability was evaluated in [19], demonstrating promising Text-to-SQL ability and robustness across various challenging scenarios. Yi et al. [22] provide a hardness parsing solution (HPSQL), in which NLQs are classified based on their complexity and routed to different fine-tuned LLMs for SQL generation.

Several recent approaches have attempted to leverage the capabilities of LLMs for Text-to-SQL tasks. A summary of the most representative comprehensive frameworks is introduced below.

SQLfuse [23], an LLM-based text-to-SQL system with four key modules: Schema Mining, Schema Linking, SQL Generation (SQLgen), and SQL Critic. The system extracts database features, maps NL queries to schema elements, and

generates SQL queries using a CoT framework. SQLgen synthesizes schema details and user intent to produce candidate SQL queries, refining them through execution feedback and constant value fixes to ensure technical correctness. Finally, the SQL Critic evaluates and selects the best query using few-shot learning and a curated knowledge base. While SQLfuse significantly improves Text2SQL performance, its complex module integration may require substantial computational resources.

Also, CoGSQL [24] achieves greater Text2SQL accuracy by integrating multiple modules, including fine-tuned components for specific tasks (coarse-grained schema linking and syntax keyword prediction) and prompt-based components for SQL composition and correction. Its complex module integration may hinder adoption in resource-constrained enterprises, highlighting a trade-off between sophistication and practical usability [25].

Guo et al. [26] augment in-context learning for Text2SQL by generating de-semanticized question skeletons using hyperbolic Poincaré embeddings, retrieving structurally similar examples, fetching reusable CoT fragments based on meta-instructions, and iteratively refining SQL queries through real-database execution feedback. Their framework (MPRAG), achieves cross-domain robustness without requiring fine-tuning.

C3 [27] is a zero-shot Text2SQL method based on ChatGPT. It uses Clear Prompting (CP), including clear layout and clear context, Calibration with Hints (CH) to guide the model in avoiding biases like unnecessary columns or incorrect SQL constructs, and Consistent Output (CO) to improve reliability by selecting the most consistent SQL from multiple generated queries. However, the solution lacks error handling to refine less precise SQL queries.

SQLAI.ai [28] is an AI-powered SQL generator designed for practical use, offering performance comparable to state-of-the-art LLMs. It can integrate with databases to improve performance, but currently lacks question analysis and error handling capabilities, requiring manual intervention for ambiguous queries and complex errors. Text2SQL.ai [29] is another AI-powered query generator. While its generation speed is fast, its accuracy is notably low, particularly when processing user queries against real-world IoT datasets.

However, current Text2SQL capabilities primarily focus on converting NL questions into executable SQL queries, enabling automated data management and retrieval. Recent research by Feretzakis et al. [30] demonstrates the necessity of embedding trust mechanisms, such as sensitivity detection, role-aware output control, and differential privacy, into LLM systems to protect sensitive data while preserving functionality. These principles are critical for securing LLM-based Text2SQL systems, where improper query generation or data exposure could violate privacy regulations. Similarly, Ojuri et al. [25] stress that privacy and security considerations must be integrated into the design of future Text2SQL systems.

Most existing LLM-based Text2SQL approaches, including the representative systems discussed above, are not designed to generate SQL statements that comply with database AC policies, nor are their systems AC-integrated. As a result, the SQL statements produced by these systems cannot be directly executed in access-controlled databases to retrieve data. In practice, users are often required to manually rewrite them to enforce their access privileges, which significantly limits the deployability of these approaches in real-world settings.

Pedro et al. [31] implement an SQL parser to analyze LLM-generated SQL queries, then semantically rewrite them by nesting the original query within a restricted subquery that applies user-specific access conditions. This method ensures that the restriction is deterministically added to the SQL before execution. However, it struggles with complex queries and lacks adaptability to new SQL structures [9]. Rubio-Medrano et al. [32] conclude that LLMs like ChatGPT 4.0 can understand, generate, explain, and correct RBAC policies, and can produce moderate code requiring minimal or no modification when provided with appropriate prompts. Xiang et al. [33] show that embedding security requirements directly into the system prompt can reduce the performance of the core agent. To address this issue, they introduce GuardAgent, the first framework that uses an LLM to enforce safety rules on other AI agents. It analyzes the target agent's behavior, safety policies, and past examples to generate executable guardrail code, which is used to check the target agent's actions before execution. However, this approach enforces security only in a post-hoc manner and does not guide the SQL generation process during inference. As a result, insecure or non-compliant SQL queries may still be produced, particularly when users and LLMs lack prior knowledge of the security constraints, leading to increased post-processing effort. The effectiveness of this approach is therefore questionable, as the initial prompt is not well optimized. In practice, GuardAgent can effectively detect insecure SQL only when provided with detailed security instructions, suggesting that incorporating security information earlier in the generation workflow could lead to better outcomes [34]. Understanding complex security constraints and incorporating them into the core agent remains a major challenge for existing language models.

Moreover, datasets such as Spider [35] are commonly used benchmarks for evaluating Text2SQL systems. However, they may not fully reflect real-world challenges, where databases often have more intricate schemas, noisy data, ambiguous queries, and AC mechanisms. These real-world complexities create challenges that go beyond what Spider is designed to represent. The papers [36], [37] demonstrate that while some LLM-based Text2SQL tools (e.g., C3 with GPT-4 [27]) excel on benchmarks like Spider and BIRD, their performance drops on real-world databases due to structural and semantic complexities. Therefore, Fürst et al. [38] assess the robustness of Text2SQL systems based on real user queries and

TABLE 1. Summary of most representative related works.

Solution	Architecture	Methods	Non-expert support	Includes security	Dataset	Metrics
SQLfuse [23]	Comprehensive	Prompting (CoT) + Fine-tuning	Yes	No	Spider	EX
COGSQl [24]	Comprehensive	Prompting (CoT)	Yes	No	Spider, Bird	Exact matching, VES
C3 [27]	Lacks error correction	Prompting (Zero-shot)	Yes	No	Spider	EX
MPRAG [26]	Comprehensive	Prompting (RAG, Few-shot, CoT)	Yes	No	Spider-family, Bird	EX, Test suite accuracy
HPSQL [22]	Lacks error correction	Fine-tuning (Hardness levels)	Yes	No	Spider	Exact matching, EX
SQLAI.ai [28]	SQLAI.ai is an AI-powered SQL generator with performance comparable to SOTA LLMs, but it lacks question analysis and error handling.					
Text2SQL.ai [29]	Text2SQL.ai is a fast AI-powered query generator, but its accuracy is low, especially with real-world IoT datasets.					
Our proposal	Comprehensive	Prompting	Yes	Yes	Real-world agricultural IoT dataset	EX, VES (Syntactics and Semantics)

their novel dataset, FootballDB, which they introduced. Pavlich et al. [39] evaluate Text2SQL models on a novel IoT dataset that includes temporal-related queries and network traffic data from a smart building environment. NLINQ [40], a domain-specific Text2SQL system designed for querying wireless mesh network performance databases, operates on real industrial network management databases and explicitly addresses practical challenges such as non-semantic table and column names, and noisy text or speech inputs by introducing semantic database views and domain-specific corrections.

In this paper, a comprehensive Text2SQL framework is proposed that integrates database-associated authority management to ensure more secure and privacy-respecting SQL generation. Unlike previous studies that often evaluate their systems on synthetically generated benchmarks, this research focuses on assessing the ability of various LLMs to handle non-expert user queries in a realistic spatio-temporal IoT dataset. The evaluation provides these models with the appropriate instructions to identify the most promising models for syntactically and semantically accurate Text2SQL generation. In addition to the widely used Execution Accuracy (EX) metric, the study uses the Valid Efficiency Score (VES) to evaluate the efficiency of valid queries, reflecting their practical usability.

Table 1 provides a comparison between the most representative solutions and the system proposed in this paper. Although these frameworks integrate SQL generation, pre-processing, and post-processing, only the methods used for SQL generation are shown in Table 1, since this is the primary focus of this work. It can be observed that none of the compared approaches incorporates database security within their Text2SQL pipelines.

IV. Text2SQL PROPOSAL

In the following, a detailed description of the architecture of the proposed system for translating natural language

queries (NLQs) to SQL queries, using Large Language Models (LLMs), is presented. The objective of this proposal is to facilitate the interaction of non-technical users with relational databases or data management systems, allowing them to extract information without the need for specialized SQL knowledge. The system architecture consists of several components designed to process NLQs accurately and robustly.

A. PROPOSED SYSTEM ARCHITECTURE

Figure 2 shows the high-level system architecture diagram. This diagram includes blocks of services that include more than one process for their execution. These components are intentionally presented in a simplified form to provide a high-level abstraction of the proposed system architecture and its overall operational flow. However, all those services, composed of several components or processes, will be detailed in the following subsections.

The proposed system is structured around a main component called “Text2SQL Service”, which is responsible for transforming a natural language query into a valid SQL query. This block receives as input natural language queries specified by the user, as well as the database schema itself as part of the context. For each query, the system follows several fundamental steps, starting with user authentication. This initial step ensures that only authorized users can access the system and perform database queries, establishing the access control necessary in a secure data environment.

Once authenticated, the system proceeds to the natural language query analysis process. In this step, the system extracts both the intentions and the specific goals of the NLQ. This initial process is crucial to ensure that the context of the query and the user’s expectations are correctly conveyed to the language model that will perform the conversion. The extracted goals and intent guide the system in identifying the relevant tables within the database that contain the required

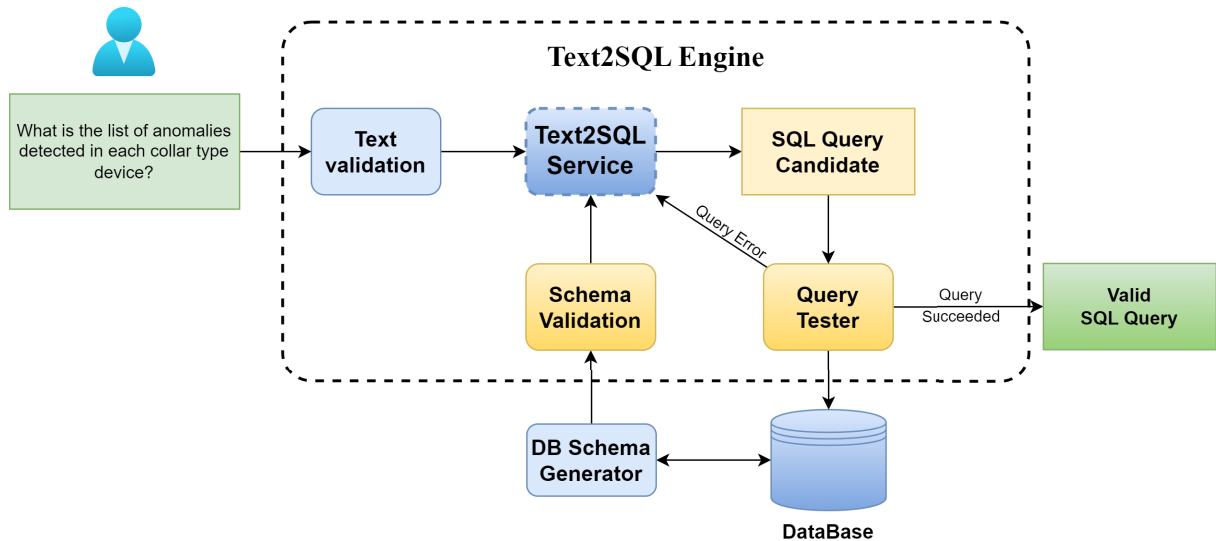


FIGURE 2. Text2SQL Engine.

information. At this point, the *Text2SQL Service* accesses the database schema structure to perform an accurate and efficient query.

With relevant information about the database structure and the query context, the system verifies the user's permissions to access the identified tables. This authorization process ensures that the generated SQL queries respect the data access restrictions, maintaining the security and privacy of the information in the system. If the user has valid permissions, the system proceeds to the candidate SQL query generation stage.

The SQL query generation is driven by an LLM, which uses the context of the original NLQ, the database schema structure, and any additional relevant information obtained in previous steps. This initial SQL query, called the "SQL Query Candidate", is generated based on the identified intents and is sent to the *SQL Query Tester* module for validation.

At this stage, the Query Tester executes the candidate SQL query in a test environment. If the query has syntax errors, inaccuracies in data retrieval, or does not meet the user's objectives, the system feeds back to the *Text2SQL Service* with additional information to expand its context and generate a refined version of the query. This iterative process allows the system to fine-tune the query to an accurate and optimal SQL version that meets the user's requirements before returning the final query result to the user interface.

B. Text2SQL SERVICE

The *Text2SQL Service* is the functional core of the system and is responsible for the effective translation of NLQs into SQL. This service is composed of several sub-components that interact in a structured flow to ensure the accuracy and relevance of the generated queries. The high-level architecture of this service is shown in Figure 3, whose components or microservices are detailed below.

- 1) **Intent and goal extraction:** Once the original NLQ has been validated, the system uses a language model

to extract the goals and underlying intent of the query. This process involves analyzing the semantic context of the query to determine the type of information the user expects to obtain. For example, if the query is "What is the list of anomalies detected in each collar-type device?", the system will identify that the user is looking for a specific list of records related to "anomalies" and "collar-type devices".

The process of extracting or identifying the underlying intent of the NLQ—while essential to the overall system functionality—is not analyzed in detail or evaluated in this paper, as it falls outside the primary scope of this study. The focus is exclusively placed on assessing the performance of LLMs in the subsequent task of translating validated NLQs into syntactically and semantically correct SQL queries.

- 2) **Identification of relevant tables:** With the objectives and intent already defined, the system uses the database schema to identify the tables that contain the data needed to answer the NLQ. This process is crucial, as it allows the system to focus the query on the sections of the database that contain the relevant information, improving the efficiency and accuracy of the query.
- 3) **Permission check:** After table identification, the system performs an authorization check to determine if the user has access to the data contained in the relevant tables. This check ensures that the generated SQL query complies with the system's security and privacy policies. Only users with the required permissions can extract the desired data; otherwise, the system notifies users about the lack of permissions to access the desired information.
- 4) **SQL query generation:** Once the previous stages have been completed, the system uses an LLM-based component to generate the SQL query. As input to this component, (i) the NLQ made by the user, (ii) the

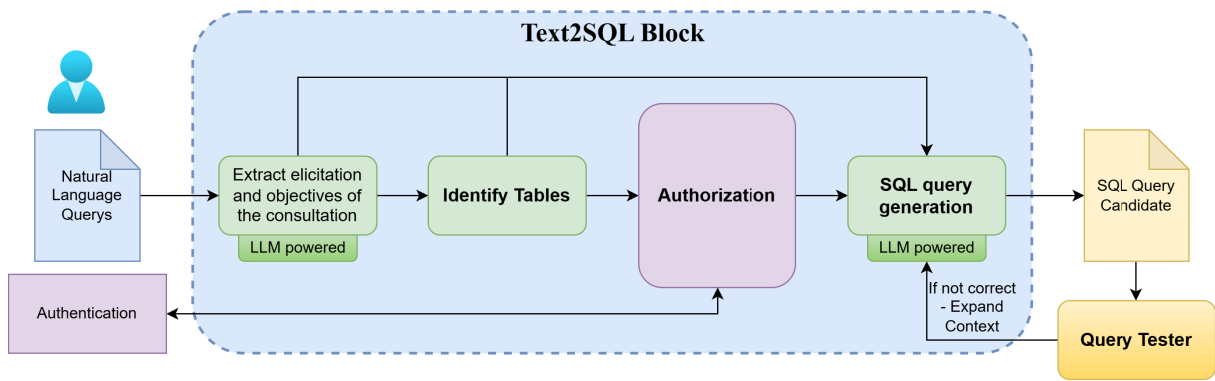


FIGURE 3. Text2SQL Service.

transcription of the intentions or objectives of the NLQ, (iii) the database schema, and (iv) the information regarding the user's specific access authorization or permissions are provided. With this information, the component generates a candidate SQL query capable of extracting the information requested by the user. The evaluation and analysis provided in this paper (Section V) focus on the selection of the most suitable LLM for integration into the proposed system.

5) Iterative testing and tuning of the SQL query:

The generated candidate SQL query is then sent to the *Query Tester module*, where it is executed in a test environment. The test environment corresponds to an isolated and read-only instance of the benchmark databases, replicating their schemas and contents exclusively for query evaluation. This controlled setting enables the safe execution of candidate SQL statements, allowing the system to verify their syntactic validity through the use of the SQL engine's ANALYZE operation and to assess their expected behavior without any risk of blocking operations, side effects, or interference with the experimental data.

If the candidate query does not retrieve the expected results or exhibits syntactic inaccuracies, the system provides the LLM-based component with enriched contextual information derived from the evaluation process. This feedback enables the model to refine its interpretation of the natural language request and adjust the subsequent query generation accordingly.

This iterative feedback and tuning approach allows the *Text2SQL Service* to not only convert NLQs into SQL queries, but also to develop an adaptive understanding of the user's intentions, improving its accuracy and relevance in data retrieval. The modular design and the use of an advanced LLM give the system flexibility and continuous learning capabilities, making it a powerful tool for non-technical users in extracting information from complex databases.

C. SYSTEM SECURITY AND AUTHORIZATION MODULES

Most existing Text2SQL systems lack integrated AC, allowing users to send any queries while providing no guarantees

regarding the security of either the generated SQL or the target databases. In real-world scenarios, databases are often governed by strict AC policies, where users may only have permission to access specific tables or columns. This introduces an additional layer of complexity to query generation.

To address the security and privacy demands of databases and LLM-powered applications, our proposed LLM-based Text2SQL system integrates with database-associated Authentication (AuthN) and Authorization (AuthZ) services. These services act as a trusted extension of the system, ensuring that only authenticated users can use it. An SQL query is considered valid only when it complies with both user intents and security specifications.

In this paper, we assume that the default AC mechanism is disabled to prevent direct requests from reaching the database, while root access is granted to the proxy, which enforces AuthN and AuthZ. Consequently, users can access authorized database data only after successful validation by the proxy. The Text2SQL system is dynamically granted the user's temporary access permissions exclusively for verifying SQL queries against the database server, ensuring compliance with the principle of least privilege and preventing attackers from exploiting the model to execute SQL injection attacks.

For our framework, we employ Role-based Access Control (RBAC), a mature security model that LLMs like GPT-4o can effectively interpret and translate into code when provided with suitable prompts [32]. To simulate real-world agricultural use cases, we define three roles: "admin", "device provider", and "farmer". The security model ensures that each identity has access only to authorized tables, columns, and rows in the database. For instance, the AC policy for the "collar" table specifies distinct data accessibility for each role (see below). Such policies, commonly used in DBMSs, enable fine-grained permission management at multiple levels [41], [42].

Access Control Policy for the collar Table

Admin users have unrestricted access to all data in the collar table.

Device providers can access only records of their assets where: `userId = collar.provider`

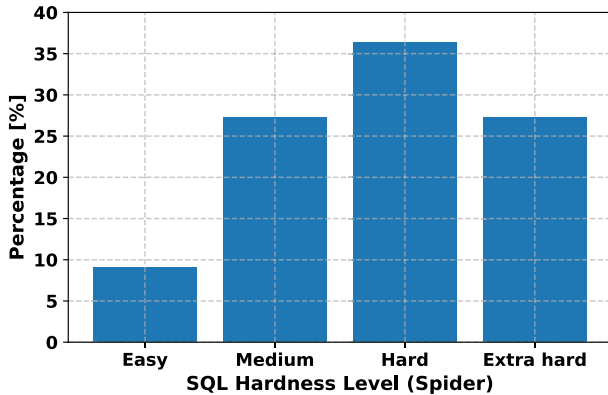


FIGURE 5. Distribution of SQL query hardness levels according to the Spider benchmark.

V. EVALUATION

This section presents a comprehensive evaluation of the proposed system to assess its effectiveness in enabling non-technical users to manage and query data through natural language interfaces. The evaluation is conducted within the domain of Precision Agriculture (PA), an application domain of IoT that provides a relevant and challenging environment for testing the system's capabilities. The rationale behind selecting this domain is twofold: (i) it exemplifies a real-world scenario where end-users, such as farmers and agricultural operators, lack expertise in data engineering but require intuitive tools for data access and management, and (ii) it offers a structured setting for systematically evaluating the translation of natural language queries into SQL statements, ensuring a clear validation of the system's design and functionality.

The evaluation process focuses on assessing the suitability of the most representative LLMs for integration into the core component of the system, responsible for translating NLQs into executable SQL statements. Rather than evaluating the complete system, this study isolates and analyzes the performance of the translation module, which receives as input user-formulated NLQs, the transcription of the intentions of these queries, and database schema information. Each LLM generates SQL candidates from the given inputs, which are subsequently validated for syntactic and semantic correctness, as well as execution efficiency. The evaluation emphasizes the models' ability to accurately capture user intent and produce queries that align with expert-defined references, thereby informing the selection of the most appropriate LLM for robust, efficient, and scalable deployment within the broader architecture.

It is important to note that response times and system scalability are not the primary focus of this study due to the limited computational resources available for experimentation. Instead, the primary objective of this work is to evaluate the system's capability to provide a functional tool that enables non-technical users to query and exploit data effectively. By structuring the evaluation in this manner, the

feasibility and effectiveness of the proposed approach in facilitating accessible data management for non-technical users in IoT-driven agricultural environments are demonstrated.

A. EXPERIMENTAL SCENARIO

For the generation of the natural language query dataset, it was initially proposed to be written directly by the end users of the system: farmers and non-technically trained agricultural specialists who are active participants in the H2020-AFarCloud European research project consortium [5]. This approach sought to capture authentic queries that reflect the language, needs, and level of technological understanding of the users the system is designed to serve.

However, during the initial collection phases, a bias was identified in the queries generated by farmers. This bias was derived in part from the explanations provided about the environment and purpose of the system, which influenced the formulations and contents of the questions. Thus, although the process was intended to ensure that the consultations were natural and spontaneous, the aforementioned limitations compromised the diversity and representativeness of the dataset.

To solve this situation and ensure a wide range of queries, an LLM in chatbot format was chosen to generate natural language questions. The implementation of this method allowed for the generation of a more diverse set of queries, with different degrees of complexity and abstraction, aligned with the range of questions that an end user could ask in a real-use context. The generated queries were then reviewed and selected according to relevance, clarity, and realism criteria to maximize the effectiveness of the dataset in evaluating the proposed system.

This approach ensures that the dataset used in the tests covers a representative spectrum of natural language queries, thus enriching the system evaluation process and facilitating its adaptation to spontaneous queries from non-technical users in agricultural environments. Table 2 represents the battery of questions generated, as well as their transcription to the intention of the supposed farmer who formulates them.

For the experiments, the transcription of each natural-language query into its underlying intent was manually produced by the research team. This manual generation ensures a precise and consistent semantic interpretation of the user request, preventing any variability that could arise from relying on an additional LLM for this task. The resulting intent statements constitute a stable reference used uniformly across all evaluated models, enabling a fair and reproducible comparison of their Text2SQL capabilities.

To characterize the complexity of the SQL queries considered in the evaluation, we classified the expert-defined reference queries according to the hardness criteria introduced in the Spider benchmark [35], which categorizes SQL queries into four levels: easy, medium, hard, and extra hard, based on structural complexity (e.g., number of selected columns, filtering conditions, aggregations, grouping clauses, and nested subqueries).

TABLE 2. Mapping of farmer’s questions to interpreted intents and data extraction requirements.

ID	Farmer’s Question	Interpreted Intent/Data Extraction
NLQ 1	“List of detected anomalies grouped by the collar in which they occurred”	The farmer needs a list of the anomalies detected in each of the collars, grouped by collar identifier.
NLQ 2	“Where are my cows at right now?”	The farmer wants to know the current location of each cow.
NLQ 3	“Are any of the cows running a fever?”	Concerned about health, the farmer asks if any cows have an abnormally high temperature.
NLQ 4	“Have any of the cows wandered off or gotten stuck?”	The farmer is checking if any cows have left the usual grazing area or haven’t moved for a while, which could indicate they’re trapped or injured.
NLQ 5	“Which cows ain’t moving around much?”	The farmer wants to know if some cows are less active than usual, which could signal illness or other issues.
NLQ 6	“How far are the cows moving each day?”	This question is about tracking the distance covered by the cows to ensure they’re grazing enough and staying active.
NLQ 7	“Did any of the collars set off an alarm?”	The farmer wants to know if any alerts have been triggered, such as unusual temperatures or activity patterns.
NLQ 8	“Are the cows staying bunched up or spread out?”	The farmer is interested in whether the herd is grazing together or dispersing over a wide area, which could indicate changes in behavior or potential problems.
NLQ 9	“Has any cow’s temperature dropped all of a sudden?”	A sudden temperature drop could be a sign of health issues like hypothermia, so the farmer wants to be notified of such changes.
NLQ 10	“Which cows are eating less, based on their movements?”	By looking at activity levels and movement patterns, the farmer wants to identify cows that might not be grazing enough.
NLQ 11	“Is any cow acting out of the ordinary, like pacing or lying down more than usual?”	The farmer is concerned with detecting unusual behavior patterns that could indicate illness, injury, or stress.

Note: The intent descriptions shown in this table were manually generated by the research team to provide a precise and consistent semantic interpretation of each natural-language query. These expert-defined intent statements served as a stable reference input for all evaluated LLMs, ensuring a fair and reproducible comparison of their Text2SQL capabilities.

Figure 5 shows the distribution of the 11 reference queries across these hardness levels. Although the evaluation set is intentionally compact, the queries span all Spider difficulty categories, with a clear predominance of hard and extra-hard queries. This distribution reflects the complexity of real-world analytical workloads in IoT scenarios, rather than aiming at exhaustive coverage. The full set of reference SQL queries is publicly available in the associated Zenodo repository [45] (DOI: 10.5281/zenodo.18036706), ensuring transparency and reproducibility.

This evaluation aims to determine the most suitable LLM for integration as the core component in the process of translating natural language queries to SQL within the proposed system. To achieve this objective, ten representative LLMs have been selected for analysis based on the metrics described in Section V-B. The choice of these models is founded on aspects such as reasoning capacity, context window size, efficient resource usage, and, in certain cases, specialization in NLQ to SQL translation. The specific reasons for selecting each model are detailed below:

- **DeepSeek R1:** [46] This model was selected for its high reasoning capacity and its demonstrated lower resource consumption compared to other LLMs with equivalent computational capabilities [46]. The evaluation intends to determine whether the complexity and wide context of DeepSeek are necessary to achieve optimal performance in translating natural language queries.

- **GPT Variants (7 versions):** Seven different versions of GPT have been included: GPT 3.0 [47], GPT 3.5, GPT 4o [48], GPT 4o mini [49], GPT o1 [50], GPT o3 mini [51], and GPT o3 mini high [51]. The inclusion of these variants addresses the need to identify the minimum requirements in terms of reasoning capacity and context window size required to satisfy the demands of the proposed system. Each version exhibits notable differences in complexity and resource efficiency, which will facilitate the identification of the “minimal” version that is sufficient for the task of NLQ to SQL translation.
- **OLLAMA SQLCoder [52] (versions 7B and 15B):** These models have been developed specifically for the task of translating natural language queries to SQL. The inclusion of both versions (7B and 15B) is intended to evaluate the impact of model size and the limited context window on the interpretation of queries from non-technical users. The analysis will assess whether task-specific specialization compensates for the restricted context size and how this approach compares with general-purpose models.

Table 3 summarizes the most relevant technical characteristics of each evaluated LLM, allowing for a visual comparison of their capabilities:

The integration of this diverse set of models enables a rigorous comparative analysis, identifying not only the performance in NLQ to SQL translation but also the balance

TABLE 3. Technical characteristics of the evaluated LLMs.

Model	Reasoning Capacity	Context Size	Window	Resource Usage	Specialization in NLQ to SQL
DeepSeek	High	Wide		Efficient relative to its complexity	General (evaluation of the necessity for such a complex model)
GPT 3.0	Moderate	Moderate		Moderate	General (baseline for minimum requirements)
GPT 3.5	Improved relative to GPT 3.0	Moderate to wide		Moderate	General (assessment of increased reasoning capacity)
GPT 4o	High	Wide		Moderate to high	General (maximum performance within the ChatGPT series)
GPT 4o mini	High (compact version)	Smaller compared to GPT 4o		Moderate	General (analysis of performance in a compact format)
GPT o1	Variable depending on design	Variable		Variable	General (evaluation of a new variant in the proposed context)
GPT o3 mini	Fast at advanced reasoning	Reduced		Low	General (compact version for efficiency analysis)
GPT o3 mini high	Great at coding and logic	Reduced		Moderate	General (evaluation of whether enhanced mini capabilities are sufficient)
OLLAMA SQLCoder 7B	Adequate for translation	Limited context		Low	High (specialized in NLQ to SQL translation)
OLLAMA SQLCoder 15B	Superior to the 7B version	Moderately larger than 7B	larger	Moderate	High (enhanced reasoning capacity while maintaining task specialization)

between model complexity and resource efficiency. This evaluative strategy facilitates the identification of the LLM that optimizes both aspects, thus representing the most suitable option for the core component of the proposed system.

The experimental scenario for evaluating the proposed system has been comprehensively described. However, several aspects concerning the hardware setup and the operational mode of the evaluated LLMs remain to be clarified. To ensure a stable experimental environment, the evaluation is divided into two phases: the translation of NLQs to SQL performed by the selected models and the subsequent performance evaluation of the generated SQL queries.

During the first phase, each LLM is provided with three distinct inputs as defined in the system: the NLQ itself, the transcription of the query’s underlying intentions, and the corresponding database schemas. For each of the eleven NLQs defined in Table 2, the experiment is executed on each model. After ten executions, each model generates ten different SQL query versions (or SQL candidates), giving a total of 110 different SQL queries per model. Furthermore, to account for the inherent variability in the responses, each generated query is executed ten times on the database. This repetitive execution allows for the calculation of average execution times, thereby mitigating the impact of potential anomalies and ensuring a robust performance evaluation.

An open-access software project developed in Python has been implemented to automate the large series of experiments. This project contains various scripts that facilitate the execution of the defined experiments and the subsequent visualization of results. The automation workflows not only streamlines the process of running hundreds of experiments

but also ensures reproducibility and consistency across all test scenarios. Aligned with the Open Science objectives of the European Union, the **source code** of the experimentation project is accessible in the **Zenodo repository** [53], with the assigned <https://doi.org/10.5281/zenodo.15181185>.

Regarding the deployment environment for the LLMs, all models—except for the Ollama SQLCoder versions, which do not provide an API for on-premise testing—have been executed via their respective provider APIs. This approach eliminates hardware constraints that could otherwise influence the performance of the models.

In contrast, the Ollama SQLCoder versions have been deployed on a dedicated server specifically configured for this purpose. This server has been maintained in a stable state throughout the experiments by disabling kernel updates, operating system updates, and any other synchronous processes that could divert system resources from the experiments. The stability of this environment was crucial to ensuring that the performance measurements reflect the true behavior of the models rather than fluctuations caused by background processes. Detailed specifications of the server are provided in Table 4. These specifications exceed the deployment requirements indicated by the model providers [52]. Moreover, the model provider specifies that the 15B version of SQLCoder is designed to operate entirely within the RAM of the host machine, requiring a minimum of 16 GB, while the experimental server used in this study is equipped with 64 GB of RAM, thus exceeding the recommended specifications. This configuration further reduces potential variability in the behavior of the Ollama-based models by ensuring a stable and controlled local execution environment comparable in consistency to provider-hosted API deployments.

TABLE 4. Equipment Used.

	Server Host
Operating System (OS)	Ubuntu server 22.04.3 LTS
Mother Board	MSI MS-7A72
Architecture	64-bit
Microprocessor	i7-7700 3.6 GHz
N ^o Cores	4 (8 threads)
RAM	64 GB
Disk	Samsung SSD 850 EVO 500GB

All LLMs were evaluated using a uniform and controlled generation configuration to ensure fairness and reproducibility across models. Specifically, the temperature was set to 0.0 to favor deterministic behavior, which is appropriate for the Text-to-SQL task where correctness and stability are prioritized over linguistic diversity. The nucleus sampling parameter was fixed to $top_p = 1.0$, effectively disabling probabilistic truncation of the token distribution. The maximum number of generated tokens was limited to 512 to prevent output truncation while accommodating complex SQL queries involving joins and nested clauses. Both *frequency_penalty* and *presence_penalty* were set to 0.0, as penalizing token repetition or encouraging lexical novelty is not suitable for structured query generation. No random seed was fixed during inference, since not all evaluated APIs consistently expose this parameter; this choice ensures methodological consistency across providers while still capturing the intrinsic non-deterministic behavior of LLMs through repeated executions.

To further support reproducibility and transparency, all prompt inputs and auxiliary experimental artifacts used in this study are publicly available in a dedicated Zenodo repository [45] (DOI: 10.5281/zenodo.18036706). The repository includes the complete prompt instances used for each NLQ, the reference SQL queries authored by domain experts, the full database schema, and the generation parameter configuration shared across all evaluated models. A *README* file provides detailed documentation of the experimental inputs and usage guidelines, enabling independent replication of the experimental setup.

The second phase of the experimentation focuses on executing the SQL queries generated by the models on a real-world database. This database contains data collected from 77 collar-type devices, deployed as part of the H2020-AFarCloud research project [5]. Although the dataset itself is not provided in this article due to ownership rights held by the participating farmers and livestock owners, Figure 6 illustrates the database schema used for storing the data. It is noteworthy that the total number of time series entries in the experimental database exceeds **813,000 samples**. The database employed is a relational PostgreSQL database [54] (version 17.0), and its deployment environment has been similarly stabilized by preventing system updates and other non-essential processes during the experiments.

The dataset used comprises real-world measurements collected by collar-type IoT devices mounted on cattle in the

```

1 CREATE TABLE collar (
2   "time" TEXT, -- Timestamp of the specific measurement, indicated as Unix
3   -- Epoch format
4   "entityName" TEXT, -- Unique ID for each collar
5   scenario TEXT, -- ID of the actuation scenario in which the collar has been
6   -- deployed
7   provider TEXT, -- ID of the device provider
8   service TEXT, -- Service provided by the device
9   type TEXT, -- Type of device (this field always has the value "collar")
10  latitude NUMERIC, -- Latitude coordinate of the geoposition of the device at
11  -- the specific timestamp of measurement generation
12  longitude NUMERIC, -- Longitude coordinate of the geoposition of the
13  -- device at the specific timestamp of measurement generation
14  altitude NUMERIC, -- Altitude coordinate of the geopositioning of the device
15  -- at the specific timestamp of measurement generation
16  geohash TEXT, -- Geohash that expresses the geoposition of the device at the
17  -- specific timestamp of measurement generation
18  "accX" NUMERIC, -- X-axis accelerometer value
19  "accY" NUMERIC, -- Y-axis accelerometer value
20  "accZ" NUMERIC, -- Z-axis accelerometer value
21  "activityAnomaly" BOOLEAN, -- Boolean for the identification of
22  -- activity anomalies
23  "distanceAnomaly" BOOLEAN, -- Boolean for the identification of
24  -- distance anomalies
25  "positionAnomaly" BOOLEAN, -- Boolean for the identification of
26  -- position anomalies
27  "locationAnomaly" BOOLEAN, -- Boolean for the identification of
28  -- location anomalies
29  "temperatureAnomaly" BOOLEAN, -- Boolean for the identification of
30  -- temperature anomalies
31  "resourceAlarm" BOOLEAN, -- Boolean for alarm identification in the
32  -- device
33  "sequenceNumber" INTEGER, -- Sequence number of the measurement
34  -- with respect to the other measurements generated by the same device
35  temperature NUMERIC, -- Temperature value generated by the device
36  CONSTRAINT key PRIMARY KEY ("entityName", "time")
37 );

```

FIGURE 6. Database schema for collar-type devices.

H2020-AFarCloud project's performance scenarios. These devices are designed for continuous livestock monitoring and behavioral analysis, capturing spatio-temporal geolocation (latitude, longitude, altitude), tri-axial accelerometer data (accX, accY, accZ), and metadata related to device operation and deployment. In addition to core sensor data, the collars register multiple anomaly indicators—such as activity, position, distance, temperature anomalies, and resource alarms—enabling the detection of atypical behavior and early signs of health issues. The accelerometric information supports retrospective analysis of behavioral patterns under varying health or environmental conditions. The data were recorded over a 16-month period between 2020 and 2021, providing a robust foundation for evaluating the system's translation accuracy and operational relevance. As noted above, due to data ownership and privacy constraints, the dataset is private and cannot be shared. Table 5 provides detailed information on the data model and the dataset used.

Together, these detailed experimental conditions define a robust and meticulously controlled scenario, ensuring that both the translation and execution phases are carried out under optimal conditions. The combination of automated experimentation, a stable execution environment, and multiple iterations for each query guarantees that the results obtained are both reliable and representative. The following section details the evaluation metrics applied to the results obtained from these experiments.

B. EVALUATION METRICS

The efficiency and performance of the selected Large Language Models (LLMs), in translating natural language

TABLE 5. Data model of the collar dataset.

Collars		
Number of Entities		77 Collar-type Devices
Number of Points		813,546
Dimensions	Time	Timestamp (Epochtime)
	Geoposition	1 - Latitude and Longitude (Float)
		2 - Geohash (String)
Structure	Semantic	Semantic information regarding data nature, identifiers, and service provider
	Primary Key	2 attributes (Temporal and Semantic Information)
		“entityName” (Unique ID for each collar)
Fields		“time” (Timestamp of the specific measurement)
		19 attributes (Spatial and Semantic Information)
	Fields	Integer: 1
		Float: 7
		Boolean: 6
		String: 5

queries into SQL, are assessed using the Valid Efficiency Score (VES) [6], which is derived from the Execution Accuracy (EX). The EX metric quantifies the correctness of the generated SQL queries by comparing their results with those of reference queries. Based on this, VES evaluates the efficiency of valid queries by integrating both accuracy and computational performance. The following subsections provide a detailed description of these metrics and their formal definitions.

1) EXECUTION ACCURACY (EX)

The *Execution Accuracy (EX)* metric quantifies the accuracy of the SQL queries generated by the model by evaluating the consistency between the obtained and expected results. Formally, it is defined as the proportion of queries where the result sets V_n (produced by the reference query Y_n) and $\hat{V}_n$ (produced by the predicted query $\hat{Y}_n$) are identical. Mathematically, it is expressed as:

$$EX = \frac{\sum_{n=1}^N \mathbb{1}(V_n, \hat{V}_n)}{N}, \quad (1)$$

where $\mathbb{1}(V, \hat{V})$ is an indicator function that returns:

$$\mathbb{1}(V, \hat{V}) = \begin{cases} 1, & V = \hat{V} \\ 0, & V \neq \hat{V} \end{cases} \quad (2)$$

and N is the total number of evaluated queries. This metric reflects the model’s ability to generate syntactically and semantically correct queries, ensuring compliance with the user’s functional requirements.

2) VALID EFFICIENCY SCORE (VES)

The *Valid Efficiency Score (VES)* [6] assesses the efficiency of valid queries, integrating both accuracy and operational performance. Unlike EX, VES considers only valid queries

($\mathbb{1}(V_n, \hat{V}_n) = 1$) and compares their relative efficiency with respect to the reference queries. The formula is defined as:

$$VES = \frac{\sum_{n=1}^N \mathbb{1}(V_n, \hat{V}_n) \cdot \mathbf{R}(Y_n, \hat{Y}_n)}{N}, \quad (3)$$

$$\mathbf{R}(Y_n, \hat{Y}_n) = \sqrt{\frac{\mathbf{E}(Y_n)}{\mathbf{E}(\hat{Y}_n)}} \quad (4)$$

where $\mathbf{E}(\cdot)$ measures the absolute efficiency (e.g., execution time, computational cost) of a query in a controlled environment, and $\mathbf{R}(\cdot)$ represents the relative efficiency. The square root in $\mathbf{R}$ mitigates the impact of extreme fluctuations, such as abnormally fast or slow executions due to external conditions. A $VES > 1$ indicates that valid queries are more efficient than the reference queries, whereas $VES < 1$ suggests the opposite.

These metrics provide a dual evaluation: EX ensures functional correctness, while VES assesses operational optimization, offering a comprehensive perspective on the model’s performance in realistic environments.

For the experimentation carried out in this paper, the calculation of the efficiency $\mathbf{E}(\cdot)$ for obtaining the VES metric is based on the “execution time” parameter of the query, as assessed in the experimental results (Section V-C) presented in this study. Given that the primary objective is to ensure that the query is efficient in terms of latency, efficiency is evaluated in terms of the execution time obtained for the established reference queries.

C. EXPERIMENTAL RESULTS

Figure 7 represents both the syntactic validity of the LLM-generated Queries, represented by the blue bars, and the results obtained from the computation of the Execution Accuracy metric (EX - equation 1), represented by the green bars.

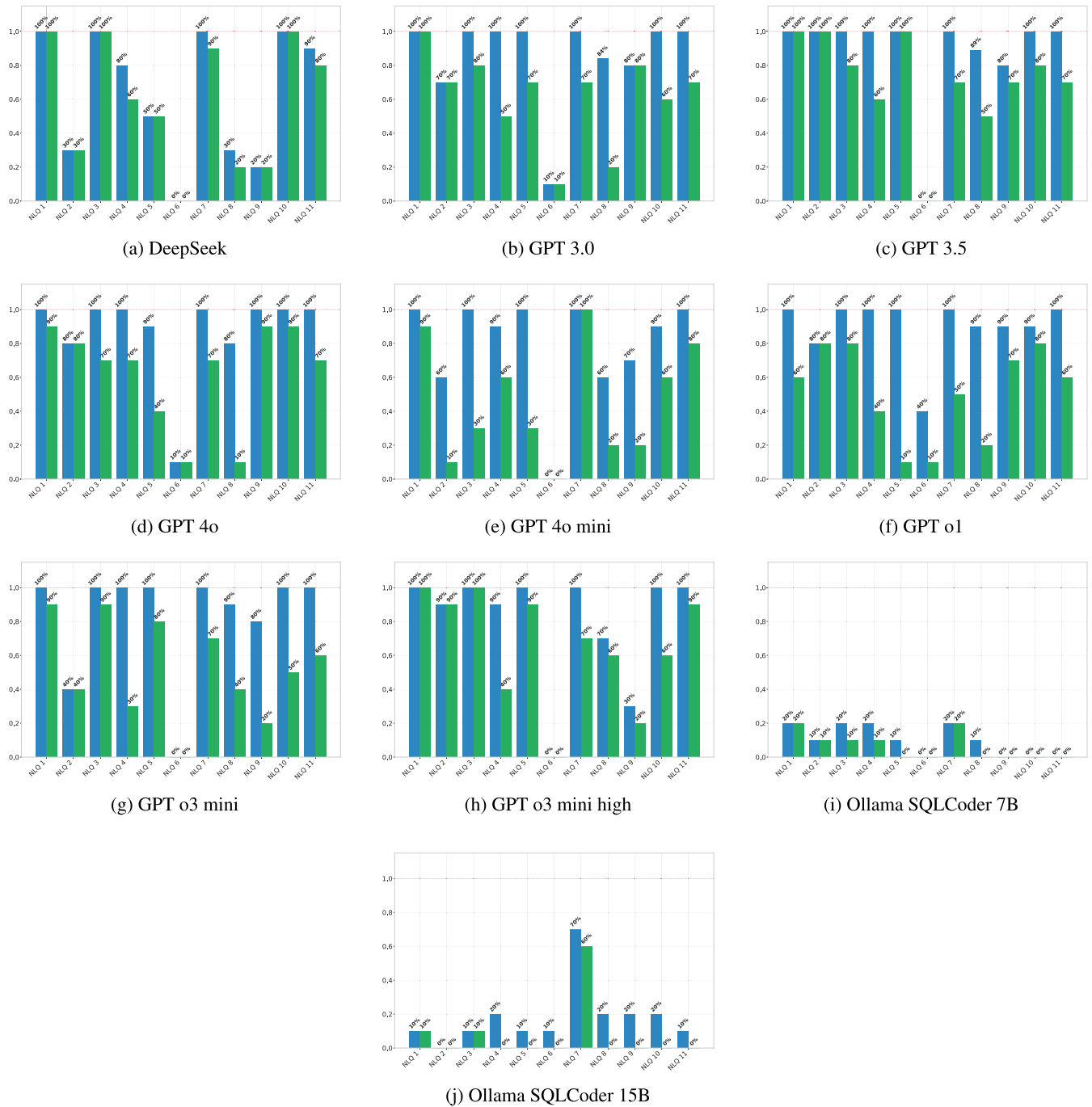


FIGURE 7. Success Rate (blue) and EX (green) per Large Language Model (LLM).

Based on the results obtained regarding syntactic validity and the EX metric, Ollama models have been discarded from the rest of the experimentation. Even been models specifically designed for the generation of SQL Queries from NLQs, these models present difficulties for the above-presented experiment due to the nature of the NLQs itself. These results remark the necessity of a wider context for the LLMs to interpret complex intents involved in the NLQs for specific application domains or use cases.

To complement the per-query EX results shown in Figure 7, we computed the average Execution Accuracy (mean $\pm$ standard deviation) for each of the eight LLMs included in the full evaluation. Each NLQ is translated into SQL ten times per model, yielding a distribution of EX values from which statistical variability can be derived. The resulting averages are presented in Figure 8, providing a compact comparison of model performance and reliability.

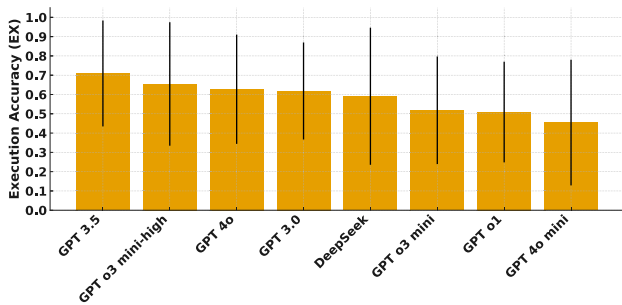


FIGURE 8. Average Execution Accuracy (EX) $\pm$ standard deviation per LLM.

Only API-based LLMs progressed to this stage, as the two locally deployed SQLCoder versions were excluded after the initial evaluation due to low EX scores. Consequently, the average EX results naturally reflect only online models, and no online/offline comparison is applicable within the scope of the final analysis.

Before presenting the VES metric for the different LLMs, Figure 9 shows the execution times obtained for the variety of queries generated by the selected LLMs and their different versions. Additionally, this figure shows the execution times of the reference queries (represented in orange), facilitating their comparison and understanding of the results of the VES metric.

The evaluation of LLMs in SQL query generation requires a rigorous study to assess both syntactic validity and semantic accuracy. Figure 10 illustrates the performance of eight LLMs across the 11 distinct NLQs presented in Table 2, quantified using the Valid Efficiency Score (VES - equation 3), a metric derived from the Execution Accuracy (EX), equation 1. The VES operates under a dual criterion: queries must adhere to formal syntactic rules and retrieve the expected dataset to contribute positively to the score. Queries that fail either condition—even if syntactically flawless—are assigned $EX = 0$, directly reducing their VES. This stringent design ensures that the metric reflects practical usability rather than formal correctness. Furthermore, when the execution of an NLQ by a given model yields a VES value greater than 1, it indicates that the generated SQL query is both syntactically and semantically valid. Additionally, a VES value exceeding 1 means that the model-generated query has outperformed the reference query formulated by the expert in terms of execution efficiency.

D. ANALYSIS

The evaluation results reveal clear performance trends across the eight large language models when assessed using the Valid Efficiency Score (VES), which jointly captures execution correctness and relative query efficiency. Rather than focusing on individual query outcomes, the analysis emphasizes aggregated behavior, robustness, and consistency across the full set of eleven NLQs.

It is important to clarify that the proposed framework does not aim to introduce a new Text-to-SQL generation

algorithm whose efficiency is directly compared with existing approaches. Accordingly, the evaluation presented in this section focuses on analyzing the suitability of representative LLMs for integration into the core component of the framework, which is responsible for translating NLQs into executable SQL statements. Within this context, the figures presented in this section compare the behavior of these LLMs by examining differences in execution correctness (EX), relative query efficiency (VES), and the stability of the performance of the SQL queries generated by each model.

Figure 11 presents the average VES values achieved by each model together with their corresponding standard deviations. This aggregated view highlights substantial differences not only in mean performance but also in variability, providing insight into each model's reliability when handling heterogeneous query formulations. Models exhibiting high average VES values combined with low dispersion can be regarded as more stable, whereas large standard deviations indicate sensitivity to specific NLQs.

A closer inspection of the results shows that performance variability differs markedly across models. GPT-4o mini exhibits the highest dispersion, despite achieving competitive VES values for certain NLQs (e.g., NLQ-4 and NLQ-11), indicating limited reliability across the evaluation set. GPT-o3 mini-high, while achieving the highest average VES among all evaluated models, also presents a relatively high standard deviation. This suggests that its strong overall performance is accompanied by notable fluctuations, with pronounced peaks in some NLQs (e.g., NLQ-7 and NLQ-11) and lower efficiency in others (e.g., NLQ-6 and NLQ-9). Consequently, high mean performance alone does not necessarily imply consistent behavior across diverse query conditions.

The impact of NLQ complexity emerges as a key factor influencing model behavior. Queries involving non-iterative aggregations or lightweight filtering operations tend to yield consistently high VES values across most models, indicating that such tasks fall within the routine capabilities of current LLMs. In contrast, NLQs characterized by ambiguous phrasing or multi-condition constraints result in greater inter-model variability. These challenges appear to stem from difficulties in accurately inferring user intent when multiple logical conditions must be satisfied simultaneously.

When considering both average performance and consistency, a clearer stratification among models becomes evident. Although GPT-o3 mini-high achieves the highest mean VES, its relatively large variability suggests that performance gains are not uniformly sustained across all NLQs. In contrast, DeepSeek achieves the second-highest average VES while exhibiting a substantially lower standard deviation. This combination indicates a more stable and predictable behavior across the evaluated queries, reflecting a stronger balance between semantic accuracy and execution efficiency.

From a system deployment perspective, such consistency is particularly relevant for applications targeting non-expert users, where predictable behavior and robustness are more

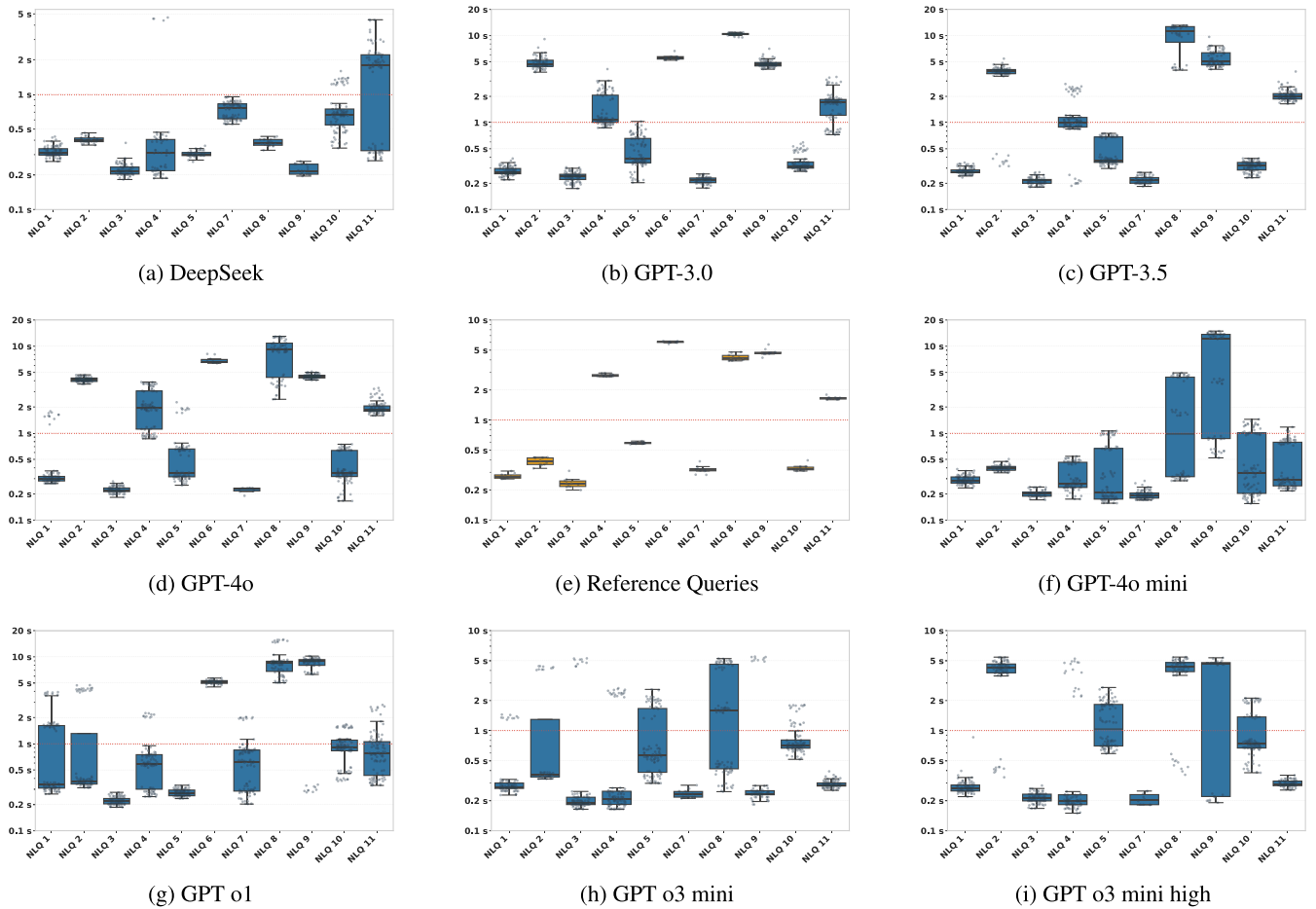


FIGURE 9. Execution Times per Large Language Model (LLM).

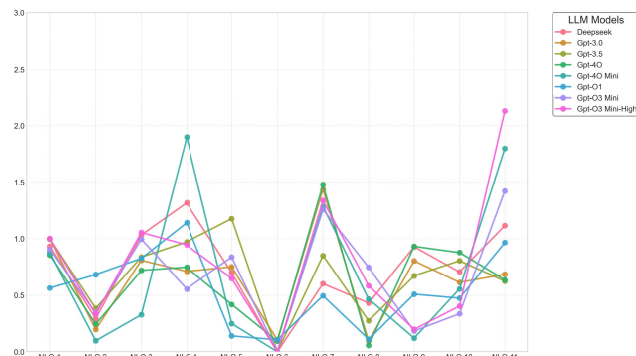


FIGURE 10. Valid Efficiency Score (VES) per LLM and NLQ ID.

critical than isolated performance peaks. Models with lower variability reduce the risk of unexpected failures or inefficient queries when confronted with diverse natural language formulations.

Overall, the analysis underscores the importance of jointly considering average performance and dispersion when selecting an LLM for practical NLQ-to-SQL systems. While models such as GPT-o3 mini-high demonstrate strong peak performance, the comparatively stable behavior of DeepSeek across varying NLQs renders it the most suitable candidate

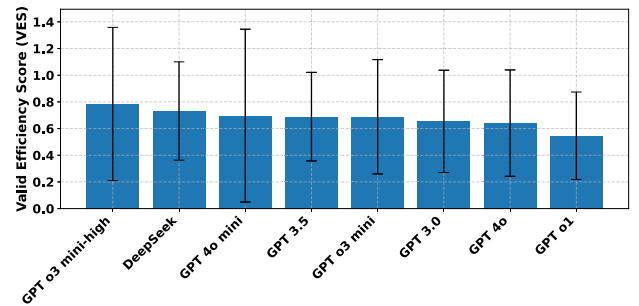


FIGURE 11. Average value of Valid Efficiency Score (VES) $\pm$ standard deviation per LLM.

for integration into the proposed system. This selection aligns with the broader objective of ensuring reliable, efficient, and semantically accurate SQL generation under real-world operating conditions.

E. PRELIMINARY SECURITY BEHAVIOR VALIDATION AND ANALYSIS

To complement the core Text2SQL evaluation presented earlier, a preliminary security behavior validation is conducted to examine how the proposed framework integrates

access control enforcement into the Text2SQL pipeline. This preliminary validation covers the complete query-generation pipeline, including interpreted intent generation, identification of required tables and columns, early permission checking, core SQL generation, policy-aware SQL rewriting, and final SQL resolution.

The objectives of this validation are threefold. First, to examine whether an instruction-following LLM can correctly interpret and apply RBAC policies when provided in structured prompts. Second, to assess whether decomposing the Text2SQL task into core SQL generation and security-aware rewriting mitigates the negative impact of security constraints on the original query generation task. Third, to empirically demonstrate that the proposed framework has the potential to effectively prevent unnecessary and unauthorized query generation at the query-generation level through early permission checking and policy-aware SQL rewriting.

A single instruction-following LLM is used to implement the Policy-Aware SQL Rewriter. The choice of the underlying model is not treated as a variable of study, as the objective is not to assess model performance but to empirically verify that the proposed framework can enforce access control constraints through rule-guided SQL rewriting. The rewriting task is primarily rule-driven and deterministic, relying on explicit RBAC policies and user context rather than open-ended generation. A comparative evaluation of different LLMs for security-aware SQL rewriting is therefore considered out of scope and left for future work.

To ensure transparency and reproducibility, the complete set of prompts used to support the security components of the proposed system, including prompts for the Intent Planner, Core SQL Generator, and Policy-Aware SQL Rewriter, is publicly available in the associated Zenodo repository [45] (DOI: 10.5281/zenodo.18036706).

Figure 12 illustrates a complete execution example of the proposed security-aware NLQ-to-SQL pipeline for the natural language query “Where are my cows at right now?” submitted by a user with the farmer role. After authentication, the authentication server returns the corresponding user attributes, including the user role and the set of accessible database components and scenarios.

The NLQ is first processed by an Intent Planner, which generates a high-level interpretation of the user request and identifies the required database components. This information is represented as a structured JSON object in which keys correspond to table names (e.g., collar) and values correspond to arrays of column names (e.g., “entityName”, “time”). In this preliminary experiment, the generated interpreted intent is consistent with the intent manually defined by the research team, indicating the model’s promising ability to capture user intent. Early filtering of irrelevant database components further reduces model hallucinations.

Based on the required tables and columns identified by the Intent Planner, the system performs an early permission check. Three user roles are assumed: Admin, Device

Provider, and Farmer. Each role is associated with a predefined set of accessible database components (e.g., database lists A, B, and C, respectively). If any of the required tables or columns fall outside the user’s authorized scope, access is denied, and the Text2SQL task is terminated. This check is implemented through explicit code-based comparison between required and accessible database components.

The Text2SQL task is then decomposed into two stages: (i) Core SQL generation, which produces a semantically correct SQL query based on the interpreted intent, required tables and columns, and database schema; and (ii) Policy-Aware SQL rewriting, which enforces RBAC by injecting row-level authorization constraints derived from the RBAC policy. The Policy-Aware SQL Rewriter operates under explicitly defined enforcement and injection rules that ensure access control constraints are added without changing the core query logic, by augmenting the Core SQL with default security conditions.

Although the Core SQL Generator produces a valid query for retrieving the most recent cow locations, the Policy-Aware SQL Rewriter injects scenario-level constraints derived from the access control policy, resulting in a final SQL query restricted to authorized scenarios only.

Queries that exceed user privileges are either rewritten to enforce the applicable access control constraints or terminated when required database components are unauthorized.

To further examine system behavior, NLQs that intentionally request data beyond the user’s permissions were tested. For example, consider a farmer authorized only for scenario AS06 who may ask: “Where are my cows of AS01 and AS06 scenarios at right now?” In this case, the Core SQL Generator produces: `SELECT DISTINCT ON ("entityName") "entityName", "scenario", "latitude", "longitude", "time" FROM collar WHERE "scenario" IN ('AS01', 'AS06') ORDER BY "entityName", "time" DESC`; The Policy-Aware SQL Rewriter transforms this query into: `SELECT DISTINCT ON ("entityName") "entityName", "scenario", "latitude", "longitude", "time" FROM collar WHERE "scenario" IN ('AS01', 'AS06') AND "scenario" IN (AUTHORIZED_SCENARIOS) ORDER BY "entityName", "time" DESC`; Consequently, only data associated with the authorized scenario AS06 is returned, preventing privilege escalation and unauthorized data access.

To facilitate deterministic server-side resolution of default security conditions into specific user attributes, the policy-aware SQL rewriting step enforces access control by conjunctively restricting the user-requested query scope with the authorized scope. This design may introduce redundant predicates (e.g., multiple IN conditions on the same attribute). Although such queries are not syntactically minimal, they remain semantically correct and preserve a monotonic security guarantee, ensuring that the rewritten query never expands the original access scope. By avoiding additional

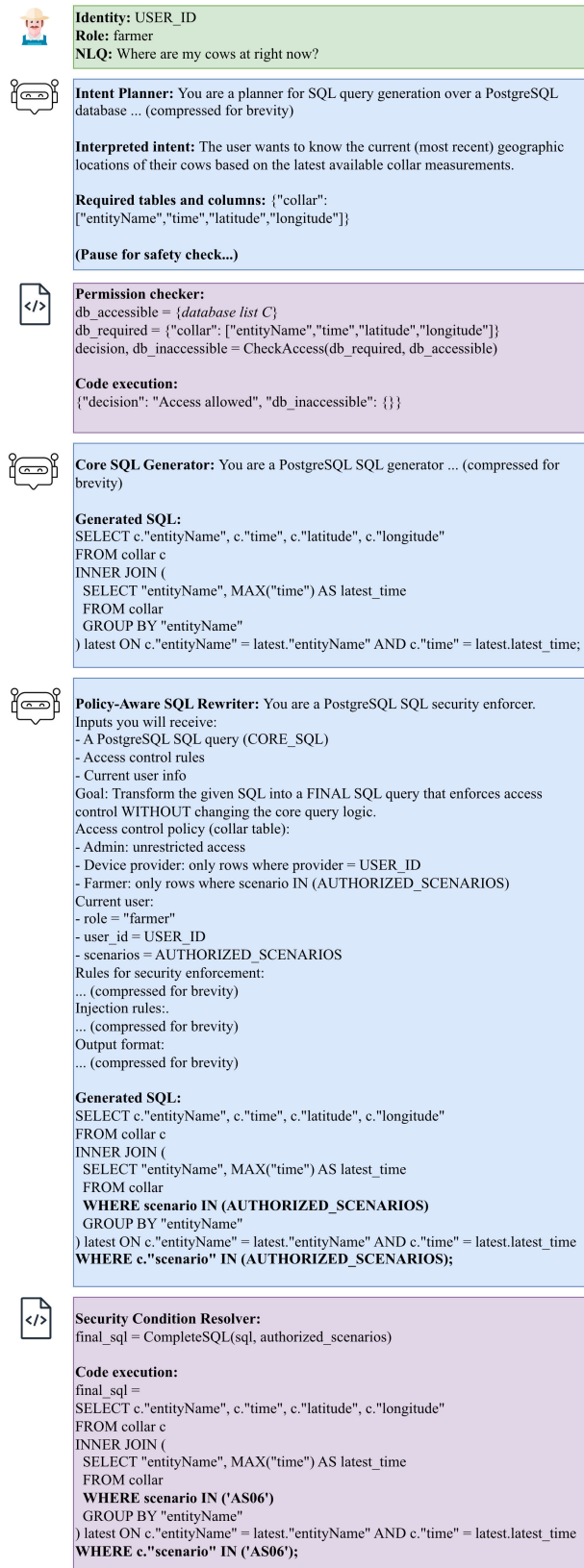


FIGURE 12. An example of policy-aware Text2SQL processing and security enforcement.

semantic rewriting logic in the LLM-based security enforcer, this approach reduces system complexity and potential error

sources. Default security conditions in the rewritten SQL are finally replaced with specific user attributes deterministically through trusted server-side code. This design represents a safe and affordable trade-off that prioritizes correctness and security over syntactic minimality.

Privacy is also enhanced. Early permission checking and final security condition resolution are performed exclusively by trusted server-side components. During SQL generation and rewriting, the LLM is provided only with generic security requirements and the user's role, while sensitive user-specific attributes (e.g., user identifiers or authorized scenarios) are abstracted using default condition values. As a result, sensitive user information is never exposed to the LLM, reducing the risk of privacy leakage while preserving correct access control enforcement.

Overall, this preliminary security behavior validation provides execution-level empirical evidence that the proposed framework strengthens traditional LLM-based Text2SQL solutions by proactively incorporating database-specific access control both before and during the SQL generation process. By enforcing access control by construction through early permission checking and policy-aware SQL rewriting, privilege violations and access leakage can be mitigated at query-generation time, rather than being addressed only at query execution. A systematic security evaluation involving a broader range of adversarial NLQs and attack patterns is left for future work.

F. THEORETICAL AND PRACTICAL IMPLICATIONS

This work advances the state-of-the-art in natural language-driven data access by presenting a comprehensive evaluation of general-purpose LLMs and models specifically designed for NLQ-to-SQL translation. Whereas previous studies have tested LLMs in isolation or used synthetic benchmarks, our study shows how a heterogeneous ensemble—from broad-context transformers to NLQ-to-SQL specialists—performs on a real-world precision agriculture dataset. The results indicate that an LLM must retain extensive context to generate accurate SQL statements.

The introduction of Execution Accuracy (EX) and Valid Efficiency Score (VES) establishes a dual-axis evaluation study. EX quantifies the semantic correctness of generated queries, while VES measures their runtime performance. Together, these metrics refine model comparisons and set a replicable standard for future research on natural language interfaces to relational databases.

The addition of a full security layer—including multi-factor authentication and role-based authorization at both table and field levels—distinguishes the proposed system as a deployable solution rather than a proof of concept. Embedding these safeguards within the translation pipeline addresses a critical gap in the current literature: secure data access for non-technical users.

Methodologically, we employ a large-scale IoT time-series dataset from the H2020-AFarCloud precision agriculture project, moving beyond toy schemas and public benchmarks.

This empirical grounding demonstrates the system's robustness in handling the heterogeneity, noise, and scale of operational agricultural deployments, providing stronger external validity than many prior efforts.

Practically, the proposed solution offers substantial benefits for domains with low technical literacy, where decision makers often lack direct IT support. By allowing direct database interaction via natural language, the system reduces latency in data retrieval and alleviates organizational bottlenecks tied to technical staff. Users gain immediate access to critical metrics—such as soil-moisture trends, crop-yield forecasts, and resource-usage statistics—promoting more precise and proactive management.

Our trade-off analysis between model sophistication and computational efficiency guides real-world deployment. Organizations with limited hardware or budgets may choose lighter, generic LLMs that meet acceptable EX and VES thresholds. In contrast, settings that demand maximum accuracy can deploy specialized and fine-tuned models at a higher resource cost.

By lowering the barrier to data exploration for non-technical users, this work also helps mitigate “inverse-skilled bias”. Granting direct access to data insights empowers professionals without formal technical training, fostering more equitable use of AI-driven analytics tools.

Despite these advances, several limitations must be acknowledged. First, our evaluation relies on a dataset specific to precision agriculture. Demonstrating applicability to other domains will require targeted evaluations and domain-specific system adaptations. Second, many IoT scenarios involve non-relational databases, highlighting the need to extend research to NoSQL interactions. Finally, ensuring the effectiveness of the security framework will depend on the definition of roles and permissions customized to each deployment context.

VI. LIMITATIONS AND FUTURE WORK

The evaluation conducted in this work focuses on the correctness and efficiency of the SQL queries generated by large language models, as measured through Execution Accuracy (EX) and Valid Efficiency Score (VES). Although no explicit user study or human-in-the-loop evaluation is included, semantic correctness is implicitly assessed through execution-based validation: a generated query is considered correct only if it returns the same result set as the expert-defined reference SQL, ensuring semantic equivalence rather than mere syntactic validity.

As a result, aspects such as robustness to open-ended linguistic ambiguity, subjective user satisfaction, or end-user interpretability of query results are outside the scope of this study. These dimensions are highly relevant for real-world deployments and represent promising directions for future work, particularly in the context of human–computer interaction and usability-oriented evaluations of natural language interfaces to databases.

While LLMs (e.g., GPT-4o) can translate natural language into SQL queries, they lack explicit training to enforce security guarantees for generated code and target databases. Current research on generating syntactically, semantically, and security-compliant SQL remains limited.

Based on a preliminary experiment, it was observed that the proposed framework demonstrates the potential to strengthen traditional LLM-based Text2SQL solutions by proactively incorporating database-specific access control both before and during SQL generation. As a result, privilege violations and access leakage can be mitigated by construction, rather than being addressed only at query execution time.

A systematic security evaluation involving a broader range of adversarial natural language queries (NLQs) and attack patterns is left for future work. In particular, future efforts will focus on further refining the system architecture and prompt design, as well as conducting more comprehensive experiments to assess and demonstrate the robustness of the proposed framework under diverse adversarial conditions.

Following the analysis presented in this paper, it has been demonstrated that through the use of LLMs, the proposed system is capable of generating SQL queries from NLQs, even when these NLQs are inherently ambiguous and require extensive context for proper understanding. Although the generated queries are syntactically valid, discrepancies have been observed between the semantic content of the dataset retrieved by some models and that produced by the reference queries. This divergence indicates that the intended meaning conveyed by the NLQ is not always faithfully preserved in the SQL translation. Such findings underscore the necessity for the validation and refinement component, defined in the proposed system architecture. This component should evaluate not only the syntactic correctness but also the semantic validity of the candidate SQL queries relative to the intrinsic intentions expressed in the NLQs. However, at present, this component is not able to provide full semantic validation.

The primary objective of the proposed system is to achieve full automation in data engineering tasks, enabling non-technical users to efficiently exploit databases and repositories generated by their devices. The observed semantic inconsistencies, however, highlight a potential shortcoming in the current approach, thereby motivating several avenues for future research. One possible solution involves incorporating a data engineering expert into the loop to manually refine the queries generated by the system. Nonetheless, this approach contradicts the overarching aim of ensuring complete autonomy for non-technical users.

An alternative strategy would be to expose the generated results to the end user, soliciting confirmation or additional clarification when necessary. While this method maintains user involvement and could potentially improve query accuracy, it may compromise the goal of full automation. Consequently, the most promising line of future work is the migration of the central translation component towards a *Mixture of Agents* (MoA) [55], [56] architecture. Under this

framework, the translation task is concurrently distributed among multiple models or instances, with each providing distinct SQL query versions. These versions would then be aggregated and subjected to an additional refinement process by a model specifically trained for this purpose. This multi-agent approach is expected to enhance semantic validity without necessitating human intervention, albeit at the cost of increased computational resource requirements.

In addition, extending, adapting, and validating the proposed framework in application domains beyond the IoT context, such as healthcare or industrial environments, constitutes an important direction for future work.

The outlined future work does not fall within the scope of the current paper; however, it provides a valuable foundation for guiding further research in this area.

VII. CONCLUSION

The findings of this study demonstrate that large language models (LLMs) possess a notable potential for the translation of natural language queries (NLQs) into SQL statements, thereby significantly lowering the barrier to database access for non-technical users. The evaluation, based on the Valid Efficiency Score (VES), reveals that model performance is intrinsically linked to the complexity and ambiguity of the input NLQs. Tasks that require straightforward data aggregations consistently yield high VES values, whereas queries involving ambiguous terminologies or the integration of multiple data relationships—such as those necessitating joins or advanced aggregations—expose inherent limitations in semantic preservation.

In comparing the diverse set of LLMs evaluated, the results indicate that advanced models exhibit clear improvements over legacy variants in terms of syntactic correctness and semantic accuracy. Among them, GPT-o3 mini-high achieves the highest average Valid Efficiency Score (VES) across the evaluated NLQs, highlighting its strong capability to generate highly accurate and efficient SQL queries. However, this performance is accompanied by a relatively high variability, indicating sensitivity to specific query formulations.

In contrast, DeepSeek R1 attains a slightly lower average VES while exhibiting substantially reduced dispersion, reflecting more consistent and predictable behavior across heterogeneous NLQs. These complementary characteristics suggest that GPT-o3 mini-high excels in peak performance, whereas DeepSeek R1 offers greater reliability and stability. From a deployment perspective, the choice between these models depends on the desired balance between maximum performance and consistent behavior, with DeepSeek R1 representing a particularly suitable candidate for scenarios where robustness and predictability are prioritized.

Furthermore, the evaluation underscores the critical importance of a validation and refinement module within the translation component. The discrepancies observed in complex cases between generated and reference queries emphasize that while syntactically correct SQL can be generated by current LLMs, ensuring semantic congruence with the intended

user query remains a challenge. This limitation calls for the development of advanced mechanisms—potentially through iterative query refinement and the incorporation of multi-agent architectures—to achieve comprehensive semantic validation without necessitating human intervention.

This work also emphasizes the importance of security-aware design in LLM-based NLQ-to-SQL systems. Rather than treating access control as a post-execution concern, the proposed framework integrates RBAC into the query-generation workflow through early permission checking and policy-aware SQL rewriting. This design enforces access control by construction, ensuring that generated SQL queries are policy-compliant before execution while preserving the core query logic. A preliminary security behavior validation suggests the potential to constrain or reject unauthorized access requests at query-generation time, even under different user roles and adversarial NLQs. These results indicate that combining LLM-based translation with system-level security enforcement is both feasible and necessary for the safe deployment of NLQ-to-SQL systems in real-world data environments.

Overall, the research supports the viability of integrating an LLM-based translation component into a broader automated system for data management. The results advocate for the adoption of the model that optimally balances translation quality, computational efficiency, and behavioral consistency as the core of the NLQ-to-SQL conversion process. Such an integration would not only facilitate accessible data extraction for non-specialist users but also contribute to the advancement of scalable, secure, and efficient human-machine interaction in data-driven environments.

In conclusion, while challenges in semantic interpretation persist, the combined use of robust evaluation metrics and targeted model selection provides a promising pathway toward the deployment of fully automated, user-friendly NLQ-to-SQL systems characterized by reliable and predictable behavior. The insights gained from this study lay a solid foundation for future research, encouraging the exploration of hybrid frameworks that merge advanced language processing techniques with domain-specific adaptations to further enhance the capabilities of LLM-driven data management solutions.

AUTHOR CONTRIBUTIONS

The contributions of the authors according to the CRediT taxonomy are: Conceptualization, formal analysis, investigation, writing—original draft, and writing—review and editing: Mario San Emeterio de la Parte, Yiting Wang, José-Fernán Martínez-Ortega, and Néstor Lucas Martínez; Data curation: Mario San Emeterio de la Parte, Yiting Wang, and José-Fernán Martínez-Ortega; Funding acquisition: José-Fernán Martínez-Ortega; Methodology and Visualization: Mario San Emeterio de la Parte and Yiting Wang; Project administration and resources and supervision: Mario San Emeterio de la Parte and José-Fernán Martínez-Ortega; and Validation: Mario San Emeterio de la Parte, Yiting Wang, and

Néstor Lucas Martínez. All authors read and approved the final manuscript.

REFERENCES

- [1] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, and X. Huang, "Next-generation database interfaces: A survey of LLM-based text-to-SQL," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 12, pp. 7328–7345, Dec. 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11160657>
- [2] K. Tiwari. (Mar. 2025). *Tiwarikajal/Seq2SQL—Natural-Language-Sentences-to-SQL-Queries*. [Online]. Available: <https://github.com/tiwarikajal/Seq2SQL—Natural-Language-sentences-to-SQL-Queries>
- [3] European Commission. (Mar. 2024). *Europe's Internet of Things Policy | Shaping Europe's Digital Future*. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/policies/internet-things-policy>
- [4] United Nations. *THE 17 GOALS | Sustainable Development*. Accessed: Feb. 19, 2025. [Online]. Available: <https://sdgs.un.org/goals>
- [5] *Aggregate Farming in the Cloud | AFarCloud | Projekt | Fact Sheet | H2020*. Accessed: Dec. 5, 2024. [Online]. Available: <https://cordis.europa.eu/project/id/783221>
- [6] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. C. C. Chang, F. Huang, R. Cheng, and Y. Li, "Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 42330–42357. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/83fc8fab1710363050bbd1d4b8cc0021-Paper-Datasets_and_Benchmarks.pdf
- [7] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," *Proc. VLDB Endowment*, vol. 8, no. 1, pp. 73–84, Sep. 2014. [Online]. Available: <https://dl.acm.org/doi/10.14778/2735461.2735468>
- [8] T. Mahmud, K. M. Azharul Hasan, M. Ahmed, and T. H. C. Chak, "A rule based approach for NLP based query processing," in *Proc. 2nd Int. Conf. Electr. Inf. Commun. Technol. (EICT)*, Dec. 2015, pp. 78–82. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7391926>
- [9] B. Qin, B. Hui, L. Wang, M. Yang, J. Li, B. Li, R. Geng, R. Cao, J. Sun, L. Si, F. Huang, and Y. Li, "A survey on text-to-SQL parsing: Concepts, methods, and future directions," 2022, *arXiv:2208.13629*.
- [10] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," 2014, *arXiv:1409.3215*.
- [11] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," 2017, *arXiv:1709.00103*.
- [12] G. Jeong, M. Han, S. Kim, Y. Lee, J. Lee, S. Park, and H. Kim, "Improving text-to-SQL with a hybrid decoding method," *Entropy*, vol. 25, no. 3, p. 513, Mar. 2023. [Online]. Available: <https://www.mdpi.com/1099-4300/25/3/513>
- [13] B. Bogin, M. Gardner, and J. Berant, "Representing schema structure with graph neural networks for text-to-SQL parsing," 2019, *arXiv:1905.06241*.
- [14] P. Yin, G. Neubig, W.-T. Yih, and S. Riedel, "TaBERT: Pretraining for joint understanding of textual and tabular data," 2020, *arXiv:2005.08314*.
- [15] Z. Cai, X. Li, B. Hui, M. Yang, B. Li, B. Li, Z. Cao, W. Li, F. Huang, L. Si, and Y. Li, "STAR: SQL guided pre-training for context-dependent text-to-SQL parsing," 2022, *arXiv:2210.11888*.
- [16] H. Li, J. Zhang, C. Li, and H. Chen, "RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2023, vol. 37, no. 11, pp. 13067–13075. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/26535>
- [17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Language Technol.*, Jun. 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>
- [18] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020. [Online]. Available: <http://jmlr.org/papers/v21/20-074.html>
- [19] A. Liu, X. Hu, L. Wen, and P. S. Yu, "A comprehensive evaluation of ChatGPT's zero-shot text-to-SQL capability," 2023, *arXiv:2303.13547*.
- [20] A. Mohammadjafari, A. S. Maida, and R. Gottumukkala, "From natural language to SQL: Review of LLM-based text-to-SQL systems," 2024, *arXiv:2410.01066*.
- [21] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, "A survey on employing large language models for text-to-SQL tasks," 2024, *arXiv:2407.15186*.
- [22] J. Yi, G. Chen, and X. Zhou, "Decoupling SQL query hardness parsing for text-to-SQL," *Neurocomputing*, vol. 621, Mar. 2025, Art. no. 129293. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231224020642>
- [23] T. Zhang, C. Chen, C. Liao, J. Wang, X. Zhao, H. Yu, J. Wang, J. Li, and W. Shi, "SQLfuse: Enhancing text-to-SQL performance through comprehensive LLM synergy," 2024, *arXiv:2407.14568*.
- [24] H. Yuan, X. Tang, K. Chen, L. Shou, G. Chen, and H. Li, "CogSQL: A cognitive framework for enhancing large language models in text-to-SQL translation," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2025, vol. 39, no. 24, pp. 25778–25786. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/34770>
- [25] S. Ojuri, T. A. Han, R. Chiong, and A. Di Stefano, "Optimizing text-to-SQL conversion techniques through the integration of intelligent agents and large language models," *Inf. Process. Manage.*, vol. 62, no. 5, Sep. 2025, Art. no. 104136. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306457325000780>
- [26] C. Guo, Z. Tian, J. Tang, S. Li, and T. Wang, "Multi-pattern retrieval-augmented framework for text-to-SQL with Poincaré-Skeleton retrieval and meta-instruction reasoning," *Inf. Process. Manage.*, vol. 62, no. 3, May 2025, Art. no. 103978. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306457324003376>
- [27] X. Dong, C. Zhang, Y. Ge, Y. Mao, Y. Gao, L. Chen, J. Lin, and D. Lou, "C3: Zero-shot text-to-SQL with ChatGPT," 2023, *arXiv:2307.07306*.
- [28] SQLAI. *Generate SQL Queries in Seconds With AI*. Accessed: Mar. 5, 2025. [Online]. Available: <https://www.sqlai.ai/>
- [29] *Convert Text To SQL With AI, in Seconds*. Accessed: Mar. 7, 2025. [Online]. Available: <https://www.text2sql.ai/>
- [30] G. Feretakis and V. S. Verykios, "Trustworthy AI: Securing sensitive data in large language models," *AI*, vol. 5, no. 4, pp. 2773–2800, Dec. 2024. [Online]. Available: <https://www.mdpi.com/2673-2688/5/4/134>
- [31] R. Pedro, D. Castro, P. Carreira, and N. Santos, "From prompt injections to SQL injection attacks: How protected is your LLM-integrated web application?" *arXiv:2308.01990*.
- [32] C. E. Rubio-Medrano, A. Kotak, W. Wang, and K. Sohr, "Pairing human and artificial intelligence: Enforcing access control policies with LLMs and formal specifications," in *Proc. 29th ACM Symp. Access Control Models Technol.* New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 105–116. [Online]. Available: <https://dl.acm.org/doi/10.1145/3649158.3657032>
- [33] Z. Xiang, L. Zheng, Y. Li, J. Hong, Q. Li, H. Xie, J. Zhang, Z. Xiong, C. Xie, C. Yang, D. Song, and B. Li, "GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning," 2024, *arXiv:2406.09187*.
- [34] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," 2023, *arXiv:2310.01798*.
- [35] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," 2018, *arXiv:1809.08887*.
- [36] E. R. Nascimento, G. García, Y. T. Izquierdo, L. Feijó, G. M. C. Coelho, A. R. de Oliveira, M. Lemos, R. L. S. Garcia, L. A. P. P. Leme, and M. A. Casanova, "LLM-based text-to-SQL for real-world databases," *Social Netw. Comput. Sci.*, vol. 6, no. 2, p. 130, Jan. 2025, doi: [10.1007/s42979-025-03662-6](https://doi.org/10.1007/s42979-025-03662-6).
- [37] L. Ma, K. Pu, and Y. Zhu, "Evaluating LLMs for text-to-SQL generation with complex SQL workload," 2024, *arXiv:2407.19517*.
- [38] J. Fürst, C. Kosten, F. Nooralahzadeh, Y. Zhang, and K. Stockinger, "Evaluating the data model robustness of text-to-SQL systems based on real user queries," 2024, *arXiv:2402.08349*.
- [39] R. Pavlich, N. Ebadi, R. Tarbell, B. Linares, A. Tan, R. Humphreys, J. K. Das, R. Ghandiparsi, H. Haley, J. George, R. Slavin, K.-K. R. Choo, G. Dietrich, and A. Rios, "Beyond text-to-SQL for IoT defense: A comprehensive framework for querying and classifying IoT threats," 2024, *arXiv:2406.17574*.
- [40] B. K. Saha, P. Gordon, and T. Gillbrand, "NLINQ: A natural language interface for querying network performance," *Appl. Intell.*, vol. 53, no. 23, pp. 28848–28864, Dec. 2023, doi: [10.1007/s10489-023-05043-z](https://doi.org/10.1007/s10489-023-05043-z).
- [41] Amazon. *Fine-grained Access Control in Amazon OpenSearch Service*. Accessed: Feb. 22, 2025. [Online]. Available: <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/fac.html>

- [42] D. A. Karunamurthy, M. Yuvaraj, J. Shahithya, and V. Thenmozhi, "Cloud database: Empowering scalable and flexible data management," *Quing, Int. J. Innov. Res. Sci. Eng.*, vol. 2, no. 1, pp. 1–23, Mar. 2023. [Online]. Available: <https://books.google.es/books?id=nc3MEAAQBAJ&pg=PA9&ots=DUIsIbYmPY&dq=loud%20database%3A%20Empowering%20scalable%20and%20flexible%20data%20management&lr&hl=es&pg=PA9#v=onepage&q&f=false>
- [43] S. Wang, T. Zhu, B. Liu, M. Ding, D. Ye, W. Zhou, and P. S. Yu, "Unique security and privacy threats of large language models: A comprehensive survey," 2024, *arXiv:2406.07973*.
- [44] E. Gupta, S. Sural, J. Vaidya, and V. Atluri, "Enabling attribute-based access control in NoSQL databases," *IEEE Trans. Emerg. Topics Comput.*, vol. 11, no. 1, pp. 208–223, Jan. 2023. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9844984>
- [45] M. San Emeterio de la Parte and Y. Wang. (Dec. 2025). *Msaneme/Text2SQL-Evaluation-Artifacts: Text-to-SQL Evaluation Artifacts (v1.0.0)*. [Online]. Available: <https://zenodo.org/records/18036707>
- [46] D. Guo et al., "DeepSeek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.
- [47] T. B. Brown et al., "Language models are few-shot learners," 2020, *arXiv:2005.14165*.
- [48] *GPT-4*. Accessed: Feb. 22, 2025. [Online]. Available: <https://openai.com/index/gpt-4/>
- [49] *GPT-4o Mini: Advancing Cost-Efficient Intelligence*. Accessed: May 22, 2025. [Online]. Available: <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>
- [50] (Sep. 2024). *Introducing OpenAI O1*. [Online]. Available: <https://openai.com/o1/>
- [51] *OpenAI O3-Mini*. Accessed: May 22, 2025. [Online]. Available: <https://openai.com/index/openai-o3-mini/>
- [52] *SQLCoder*. Accessed: May 11, 2025. [Online]. Available: <https://ollama.com/library/sqlcoder>
- [53] M. San Emeterio de la Parte. (Apr. 2025). *Grys-UPM/LLM-Based-HMI_evaluations: LLM-Enabled-HMI_llms-Experiments*. [Online]. Available: <https://zenodo.org/records/15181186>
- [54] (Feb. 2025). *PostgreSQL 17.4 Documentation*. [Online]. Available: <https://www.postgresql.org/docs/17/index.html>
- [55] J. Wang, J. Wang, B. Athiwaratun, C. Zhang, and J. Zou, "Mixture-of-agents enhances large language model capabilities," *ArXiv*, vol. abs/2406.04692, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:270357878>
- [56] S. S. Bavirithi, D. P. Sreya, and T. Poojitha, "Comparative analysis of mixture-of-agents models for natural language inference with ANLI data," *Natural Lang. Process. J.*, vol. 11, Jun. 2025, Art. no. 100140. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2949719125000160>



MARIO SAN EMETERIO DE LA PARTE received the international Ph.D. degree (Hons.) in telecommunications engineering from the Universidad Politécnica de Madrid (UPM), Spain, in 2024. His doctoral research titled "Contributions to Data Engineering in Spatio-Temporal Semantic Data Management for IoT," focused on advancing intelligent data management and processing frameworks for large-scale distributed systems. He is a Professor with the Department of Telematics and Electronic Engineering, UPM; and a Researcher and the Scientific Project Manager with the Next-Generation Networks and Services Research Group (GRyS). He has actively contributed to several international, European, and national projects, including H2020-AFarCloud, H2020-DEMETER, and HORIZON EUROPE-Chips JU NextArc, which focus on developing data-driven intelligent solutions for complex environments. His research interests include data science, data engineering, artificial intelligence (AI), the Internet of Things (IoT), and cyber-physical systems (CPS), with particular emphasis on spatio-temporal semantic data management and distributed semantic middleware architectures.



management, access control, Internet of Things, cloud computing, and edge computing.

YITING WANG received the M.S. degree in the Internet of Things from the Universidad Politécnica de Madrid (UPM), Madrid, Spain, in 2021, where she is currently pursuing the Ph.D. degree in systems and services engineering for the information society. She is a Predoctoral Researcher with the Department of Telematics and Electronics and a member of the NextGeneration Networks and Services Research Group (GRyS), UPM. Her research interests include identity



JOSÉ-FERNÁN MARTÍNEZ-ORTEGA received the Ph.D. degree in telematic engineering from the Universidad Politécnica de Madrid (UPM), Spain, in 2001. He is a Full Professor with the Department of Telematics and Electronic Engineering (DTE), Universidad Politécnica de Madrid (UPM), and the Head of the Next-Generation Networks and Services Research Group (GRyS). He was a Guest Professor with Mälardalen University (MDU), Sweden, from August 2013 to 2018, following

an invitation as an International Expert. He has authored several national and international publications included in the Science Citation Index in his interest areas, and responsible of several international, European, and national Projects, including research contracts with different IT companies. His main interest areas and expertise are ubiquitous computing, the Internet of Things (IoT), networked cooperating agent, and cyber-physical systems (CPS).



NÉSTOR LUCAS MARTÍNEZ received the Ph.D. degree in telecommunications engineering from the Universidad Politécnica de Madrid (UPM), Spain, in 2021. He is an Associate Professor with the Department of Telematics and Electronic Engineering (DTE), UPM, and a Researcher with the Next-Generation Networks and Services Research Group (GRyS). He has authored several international publications indexed in the Science Citation Index in his areas of expertise. He has

also participated in multiple international research projects on intelligent and adaptive systems, such as the Horizon 2020 projects DEMETER and AFarCloud, and previously in SWARMS, ACCUS, DEMANES, and WoO. His research focuses on the design and application of adaptive models and artificial intelligence (AI) techniques in middleware systems for mission management in autonomous agents. He is particularly interested in intelligent middleware that supports coordination and cooperation among distributed agents operating in dynamic environments. His work spans the Internet of Things (IoT), cyber-physical systems (CPS), decision support systems, and the edge-fog-cloud continuum.

...